

PPCA: Privacy-Preserving Continuous Authentication Scheme With Consistency Proof for Zero-Trust Architecture Networks

Tianqi Wu, Guyue Li[✉], *Member, IEEE*, Jiaheng Wang[✉], *Senior Member, IEEE*,
Bin Xiao[✉], *Fellow, IEEE*, and Yubo Song[✉]

Abstract—Continuous authentication (CA) has been widely applied by network service providers to verify user identities in finance, healthcare, and e-commerce fields. However, in next-generation networks, CA faces the risk of user privacy leakage due to its dependence on a verifier-centric authentication model, and verifiers may not always be trustworthy, particularly in zero-trust architecture networks. Existing privacy protection schemes face challenges in solving this problem because these schemes will weaken the linkability of context requests, leading to difficulties in consistency checks for fine-grained CA. To fill the gap, this article proposes a privacy-preserving CA (PPCA) scheme by incorporating anonymous self-sovereign identity and fine-grained CA. Specifically, PPCA exploits subset proof to enable users to reveal only the minimum necessary identity data for selective disclosure. To support fine-grained CA, we construct a new consistency proof for the anonymous user to prove that the different credentials are bound to the same attributes set, where the user is responsible for deciding whether to send the consistency proof. PPCA is formalized, defined, and constructed based on BLS signatures, Set Commitment, and Sigma Protocol. The security analysis shows that PPCA is correct and sound and supports user anonymity and credential consistency at the same time. The performance evaluation shows that PPCA requires only minimal additional time cost, achieving an optimal balance between security and efficiency.

Index Terms—Anonymous credential system, continuous authentication (CA), zero-trust architecture (ZTA).

I. INTRODUCTION

CONTINUOUS authentication (CA) is a security paradigm that continuously monitors and evaluates

Received 11 November 2024; revised 12 January 2025 and 25 January 2025; accepted 28 January 2025. Date of publication 3 February 2025; date of current version 23 May 2025. This work was supported in part by the Natural Science Foundation on Frontier Leading Technology Basic Research Project of Jiangsu under Grant BK20222001; in part by the National Natural Science Foundation of China under Grant U22B2006; and in part by the Key Technologies R&D Program of Jiangsu under Grant BE2022068-3. (Corresponding author: Guyue Li.)

Tianqi Wu, Guyue Li, and Yubo Song are with School of Cyber Science and Engineering, Southeast University, Nanjing, 211189 China and also with Purple Mountain Laboratories, Nanjing, 211111, China (e-mail: wutianqi1997@seu.edu.cn; guyueleee@seu.edu.cn; songyubo@seu.edu.cn).

Jiaheng Wang is with the National Mobile Communications Research Laboratory and the Frontiers Science Center for Mobile Information Communication and Security, Southeast University, Nanjing 211189, China, and also with the Purple Mountain Laboratories, Nanjing 211111, China (e-mail: jhwang@seu.edu.cn).

Bin Xiao is with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong, China (e-mail: b.xiao@polyu.edu.hk).

Digital Object Identifier 10.1109/JIOT.2025.3537980

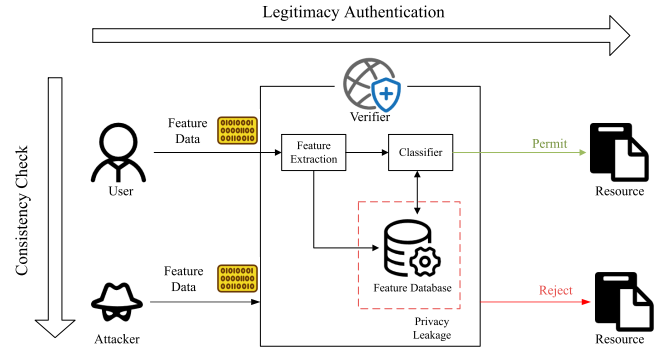


Fig. 1. Feature-based CA.

user behavior and the environment to verify user identity consistently. Unlike traditional one-time authentication, CA operates throughout the entire user session to ensure the user's identity remains trustworthy. CA requires attribute-based authentication to verify user identities. It also requires fine-grained consistency authentication to ensure that user request contexts come from the same source to avoid hijacking. It analyzes data streams from devices, such as sensor data [1], touch dynamics [2], [3], and physical location [4], to estimate the current user's legitimacy in real time. CA is widely applicable, including fraud prevention in financial services [5], healthcare [6], and e-commerce [7]. The advantages of CA lie in its ability to achieve higher security through continuous monitoring, enabling the timely detection and prevention of unauthorized access.

In the current landscape of cyber threats, various types of network attacks emerge constantly. Traditional perimeter-based security models are gradually being outpaced by zero trust architecture (ZTA). ZTA emphasizes the access control principle of “never trust, always verify,” which aligns closely with the concept of CA. As a result, CA is often applied in applications based on ZTA. For example, zero trust network access (ZTNA) is a network security framework that hides network resources from external access and only grants authenticated users permission to access specific resources [8].

To demonstrate the motivation of CA, we describe the process of WiFi authentication in a public place, such as a cafe or hotel. In these scenarios, the WiFi access point decides whether the users can be accepted according to the access policy. For example, users need to prove that they have purchased the WiFi service from a cafe or have checked in

at a hotel front desk. Moreover, the other fine-grained task of the WiFi access point is to calculate the charges of users based on their total usage time. Due to the movement, users may connect to the same network multiple discrete times, and attackers may exploit a legitimate user's previous session token to hijack the user's identity and impersonate the legitimate user when connecting to a WiFi access point. For this reason, the WiFi access point needs to conduct a consistency check to authenticate the user continuously. To sum up, CA needs to accomplish the following two goals.

- 1) *Legitimacy Check*: Verifying that the users satisfy the access policy.
- 2) *Consistency Check*: Verifying whether requests come from the same user and preventing hijacking for fine-grained CA.

Privacy Leakage: In the above case, CA is verifier-centric, which implicitly assumes that the verifier is trustworthy. However, in practical application scenarios, the verifier is not always trustworthy. First, centralized verifiers may have vulnerabilities. Attackers could exploit flaws in the verifier system to access users' private information. Second, the verifier itself could be malicious. A malicious verifier could misuse the private information provided by users for various purposes, such as selling the information to third parties, committing identity theft, or manipulating user behavior. In the next-generation network, every participating node is viewed as untrustworthy [9]. This perception arises from the network's decentralized structure, which permits nodes to operate without requiring authentication or centralized control. Due to the anonymity and openness of these nodes, their true identities and motivations can hardly be reliably determined, posing increased security and trust challenges for the network. Such untrusted networks cause significant privacy leakage issues for existing CA schemes. Privacy leakage refers to the unauthorized acquisition or disclosure of personal information [10]. In the previous goal of CA, either a legitimacy check or consistency check, users are required to provide some personal information for authentication. The abuse of collection, storage, and analysis of such information may expose user privacy, such as fingerprints, facial features, keyboard input patterns, and mouse movement. As shown in Fig. 1, in existing CA schemes, the verifier needs to collect the user's characteristic data, extract features, establish a characteristic database, and determine the user's identity legitimacy through a classifier. Since users lack the authority to determine which data the verifier collects, how it is utilized, and who has access to it, the verifier may share the data with a third party without users' permission, which could lead to further privacy leakage. Additionally, the transmission of the data over networks introduces the risk of eavesdropping or interception, increasing the risk of privacy leakage.

Therefore, the goal of this article is to explore a privacy-preserving CA scheme that supports self-sovereign identity (SSI) and fine-grained CA in the next-generation network. Here, SSI will allow legitimacy checks to be conducted by verifiers without exposing users' privacy, while fine-grained CA will help realize consistency checks even when users remain anonymous. Unfortunately, we can hardly realize

SSI and fine-grained CA simultaneously based on existing privacy protection schemes [11]. To realize SSI, these schemes require to achieve anonymity between requests to preserve user anonymity. However, to ensure that the requests come from the same user, fine-grained CA asks verifiers to continuously authenticate user identities through consistency proof, which will inevitably establish linkability between requests. Therefore, a conflict exists between the anonymity required by SSI and the linkability needed by fine-grained CA, which makes it challenging to apply privacy protection schemes directly to realize privacy-preserving CA.

Idea: First, we exploit subset proof for each legitimate check without sacrificing user's privacy. The user will be accepted if the set of attributes it possesses is proven to satisfy all the access requirements of the verifier. The user's attributes are only selectively disclosed as the verifier cannot obtain any other attributes of the user during this process. Additionally, the verifier is prevented from obtaining user identity by randomizing user's credentials. To this end, the privacy of user attributes and identity are both protected. Second, for consistency check, we construct a new proof that can prove whether two different credentials are bound to the same set of attributes. When the user intends to conduct CA, it can send this consistency proof to the verifier. In addition, since this proof is based on the Sigma protocol, it will not reveal the user's set of attributes or specific identity during this process.

Our consistency check can ensure SSI and fine-grained CA at the same time by seeking a compromise of linkability and anonymity. In this mechanism, users are allowed to decide whether to send the consistency proof to establish linkability between context requests. Specifically, if the user prefers not to establish such linkability, it can refuse to send a consistency proof, preventing the verifier from linking it to some specific requests as shown in Fig. 2(a). Conversely, if the user prefers to access specific resources and services from the verifier, it can send a consistency proof to enable the linkability while keeping privacy-preserving as shown in Fig. 2(b).

The existing schemes, such as linkable ring signature (LRS) [12], [13], [14], linkable group signature [15], [16], and traceable ring signature [17], force users to provide linkability all the time, which might weaken user anonymity. Compared to these schemes, our approach allows users to decide whether to establish such linkability by themselves, maximizing users privacy protection. By submitting randomized credentials to the verifier, users can prove the legitimacy of their identity. Meanwhile, users can optionally decide whether to submit a consistency proof to the verifier to pass the consistency check. However, how to design the consistency proof is also challenging, because the additional information in consistency proofs may lead to the leakage of users privacy information, and attackers might also use the proofs to bypass the authentication of verifiers and impersonate the legitimate identity of users. Therefore, the proposed method should be well-designed to keep user privacy-preservation and the authenticity of user identity.

Contribution: For these reasons, our proposed scheme has to realize anonymity and linkability at the same time, allowing

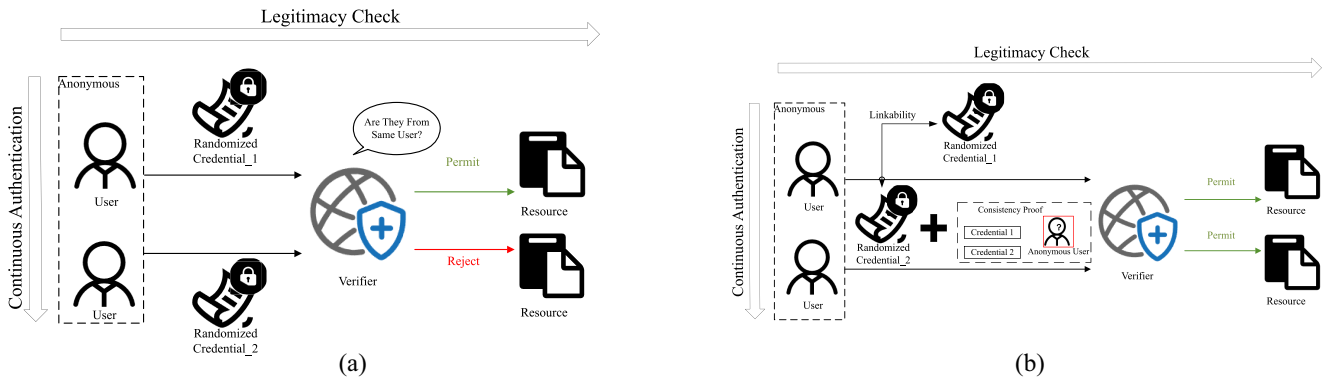


Fig. 2. PPCA with consistency proof to enable linkability. (a) Users refuse to send consistency proof for anonymity. (b) Users decide to send consistency proof for CA.

users to autonomously choose to establish linkability while always maintaining the anonymity and privacy of user identity information. Only when the user wants to access specific resources will it send the consistency proof to establish the linkability between credentials, and nobody can hijack their identity. Even if the user decides to send the consistency proof, the verifier can only determine that the credentials come from the same user and cannot determine the user's real identity.

Our contributions are summarized as follows.

- 1) We propose privacy-preserving CA (PPCA), which combines subset proof and a new consistency proof for users to complete the legitimate check and consistency check goals of CA. This mechanism achieves a compromise of linkability and anonymity and it can realize SSI and fine-grained CA without sacrificing user's privacy.
- 2) We provide a formal definition of PPCA and outline its security goals from the view of both the verifier and the user. Under this framework, PPCA can ensure the consistent verification of user identities within CA while safeguarding the anonymity and privacy of user identity information.
- 3) We propose a construction of PPCA based on BLS signature, Set commitment, and Sigma Protocol. We prove that our construction can meet the security requirements of correctness, soundness, anonymity, and linkability with a negligible efficiency loss.

II. RELATED WORKS

In this section, we present works relevant to our schemes, focusing on CA and ACS.

A. Continuous Authentication

The primary benefit of CA over the traditional static authentication scheme is that it provides an additional protection layer of security, safety, and convenience [18]. Static authentication schemes provide security only at specific moments, which are easier to imitate, and once a fraudulent user gains access, there is no way to stop them. In contrast, CA operates constantly. It simply observes user behavior and activity, identifying them at a high frequency using complex evaluation methods. The user is constantly monitored in one

or more ways, in search for signs of unauthorized access [19]. Currently, researchers are focusing on what kind of characteristics can be used to authenticate the users, such as biometric characteristics [20], [21], behavioral characteristics [22], [23], and mixed characteristics [24], [25].

However, existing works on CA are verifier-centric, ignoring the fact that untrusted or insecure verifiers can potentially lead to the leakage of user privacy. Acar et al. [10] is the first to discuss privacy protection in CA. They propose a lightweight, privacy-preserving, and secure CA protocol. It is a password-based key exchange mechanism, that continuously authenticates users based on their biometrics in a privacy-aware manner. However, their work focuses on designing key exchange protocols to prevent user privacy data from being leaked over public channels. They do not take into consideration the possibility that the verifier itself might be untrustworthy. Moreover, their work mainly targets privacy protection for biometric data, limiting the scalability of their solution in other CA scenarios.

B. Anonymous Credential System

Since the first ACS proposed by Camenish and Lysyanskaya [26], ACS has various definitions. One widespread view is that an anonymous credential is a data structure that encapsulates certain information about its holder along with a signature on that information by another entity [27] and the system using anonymous credentials to realize authentication is ACS. This technique allows users to prove their identity, membership in a group, or any other arbitrary attribute without revealing their personal information. ACS is similar to the public key infrastructure (PKI) certificate system but with a focus on protecting user privacy. Emphasizing access control while preserving user privacy, they are widely used in blockchain identity management systems to achieve self-sovereign identity. ACS has been widely used in building privacy protection in smart cards [28], health management [29], cloud computing [30], online social networks [31], vehicular ad-hoc networks [32], and decentralized finance [33]. Currently, researchers are focusing on designing ACS with additional security properties for specific scenarios, such as aggregated ACS for multiauthority

system [34], [35], revocable ACS for VANETs [32], [36], and delegatable ACS for anonymity in credential delegation [37].

Although ACS has been widely used for user privacy-preservation, it is still challenging for ACS to be applied to CA [11]. ACS aims to protect user identity privacy, preventing unauthorized access or leakage of personal information. It is required to realize anonymity between requests. Conversely, CA is for the verifiers to authenticate user identities continuously, ensuring the system can verify the legitimacy of user identity throughout the entire session. In other words, CA needs to conduct consistency proof to establish linkability between requests to ensure that the requests come from the same user. Therefore, there is a conflict between the anonymity in privacy protection schemes and the linkability in CA, which makes it challenging to apply privacy protection schemes directly to CA. Therefore, there is a need to propose a new solution that strikes a balance between them and achieves SSI and fine-grained CA.

C. Schemes Supporting Anonymity and Linkability

In this article, we aim to protect users' privacy in CA by achieving both anonymity and linkability simultaneously. Some existing schemes, such as LRSs [12], [13], [14], have already provided these properties, but they are not suitable for CA.

LRS is a special type of ring signature which supports linkability. It allows signatures from the same signer to be linked while maintaining user anonymity. To achieve linkability, LRS schemes usually construct tags generated by the users' private key, which partially binds the signatures to the users who sign the signatures [13]. Signatures from the same user will have tags that are correlated, which can be used to establish linkability.

However, applying LRS schemes directly to CA is challenging. First, there is a fundamental conflict between LRS and CA in authentication. LRS essentially proves what a user has to authenticate their identity, whereas CAs authenticate a user based on who they are. Authentication schemes based on LRS still tend to use tokens for authentication, which carries the risk of hijacking. If a user's token or private key is compromised, attackers can easily impersonate the user on other devices. Second, the linkability provided by LRS is not optional but mandatory. A curious verifier can forcibly check the linkability between any two credentials to determine whether they originate from the same user. Users prefer to offer this linkability only when necessary, rather than being forced.

In addition to LRS, there are other similar schemes, such as linkable group signatures [15], [16] and traceable ring signatures [17]. These schemes face similar challenges as LRS when applied to CA.

To address these problems, we propose PPCA by introducing the ACS into the CA scenario. PPCA allows users to decide whether to send a consistency proof for consistency check in CA to establish linkability while keeping privacy-preservation. It also enables the verifier to continuously authenticate the prover's identity. Compared to existing

research [10], this article focuses more on establishing an anonymous authentication mechanism for CA in scenarios where the verifier may be untrustworthy. Additionally, we aim to provide a more general solution that can adapt to various scenarios.

III. PRELIMINARIES

In this section, we introduce the building blocks of the construction of PPCA, including the bilinear map, BLS signature, and set commitment, which will be used in Section V.

A. Bilinear Map

Assume that $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ are multiplicative cyclic groups of prime order q . g_1 and g_2 are the generators of $\mathbb{G}_1$ and $\mathbb{G}_2$. There is a bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ has the following properties.

- 1) *Bilinear*: For $\forall a, b \in \mathbb{Z}_q$ and $u \in \mathbb{G}_1, v \in \mathbb{G}_2$, the bilinear map e has the relation: $e(u^a, v^b) = e(u, v)^{ab}$.
- 2) *Nondegenerate*: $e(g_1, g_2) \neq 1$.
- 3) *Computable*: For $\forall a, b \in \mathbb{Z}_q$ and $u \in \mathbb{G}_1, v \in \mathbb{G}_2$ there exists an efficient method to compute the bilinear map $e(u^a, v^b) = e(u, v)^{ab}$.

In this article, bilinear map is the cryptographic foundation of other building blocks and our proposed PPCA.

B. BLS Signature

The BLS signature is proposed by Boneh et al. [38] and they proved the strong unforgeability property assuming the co-CDH assumption.

The BLS signature scheme consists of the following tuple of algorithms ($KGen, Sign, Verify$). Let bilinear group $BG = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$ is public parameters, and $H : \{0, 1\}^* \rightarrow \mathbb{G}_1$ be a random oracle which is also public.

- 1) $KGen(1^\kappa)$: Sample $s \leftarrow_{\$} \mathbb{F}_p$ and set verification key as $vk = g_2^s$ and the signing key as $sk = s$.
- 2) $Sign(m, sk)$: Output $\sigma = H(m)^{sk}$.
- 3) $Verify(vk, m, \sigma)$: Output 1 if $e(g_2, \sigma) = e(vk, H(m))$. Otherwise, output 0.

BLS is used to construct a confidential signature in PPCA, allowing users to register without disclosing any private information about their identities. It helps to decrease potential privacy leakage risks during registration.

C. Set Commitment

Set Commitment is initially introduced to commit to sets and besides ordinary opening also supports opening of subsets [39]. In this section, the foundation for Set Commitment is briefly presented.

A Set Commitment scheme SC consists of five algorithms ($Setup, Commit, Open, OpenSubset, Verify$) as follows.

- 1) $Setup(1^\kappa, 1^t)$: This probabilistic algorithm takes as input a security parameter κ and an upper bound t for the cardinality of committed sets. It returns public parameters pp

$$\alpha \leftarrow_{\$} \mathbb{Z}_p, pp = \{g_1^{\alpha^i}, g_2^{\alpha^i}, BG\}$$

where α is a toxic waste, and $BG = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$.

- 2) *Commit*(pp, S): This probabilistic algorithm takes as input the public parameters pp and a nonempty set S . It outputs a commitment C to set S and opening information O

$$f_s(X) = \prod_{s \in S} (X - s) = \sum_{i \in [n]} f_i X^i \quad (1)$$

$$C = \left(\prod_{i \in [S]} g_1^{\alpha^i f_i} \right)^\rho, \rho \leftarrow \mathbb{Z}_p. \quad (2)$$

The polynomial $f_s(X)$ is used to encode the elements in set S . The commitment C can commit to the set S by committing to the polynomial $f_s(X)$.

- 1) *Open*(pp, C, S, O): This deterministic algorithm takes the public parameters pp , a commitment C , a set S , and opening information O . If O is a valid opening of C to S , it outputs 1, and 0 otherwise.
- 2) *OpenSubset*(pp, C, S, O, T): This (deterministic) algorithm takes the public parameters pp , a commitment C , a set S , opening information O and a nonempty set T . It returns $\perp$ if $T \subset S$; else it returns a witness W for T being a subset of the set S committed to in C

$$f_T(X) = \prod_{t \in T} (X - t) = \sum_{i \in [|T|]} f_i X^i \quad (3)$$

$$f_{S/T}(X) = \prod_{s \in S, s \notin T} (X - s) = \sum_{i \in [n-|T|]} f_i X^i \quad (4)$$

$$W = C_{S/T} = \left(\prod_{i \in [n-|T|]} g_1^{\alpha^i f_i} \right)^\rho. \quad (5)$$

Similar to the steps in Commit, elements in set S but not in set T is encoded into a polynomial $f_{S/T}(X)$. The witness W can be considered as a commitment to the set S/T .

- 1) *Verify*(pp, C, T, W): This deterministic algorithm takes as input the public parameters pp , a commitment C , a nonempty set T , and a witness W . If W is a witness for T being a subset of the set committed to in C , it outputs 1, and 0 otherwise

$$e \left(W, \prod_{i \in [|T|]} g_2^{\alpha^i f_i} \right) \stackrel{?}{=} e(C, g_2). \quad (6)$$

Set Commitment is used to construct the subset proof in PPCA. The subset proof allows users to prove that they meet the verifier's access requirements without revealing any additional information about their real identity. It supports the legitimate check in CA while preserving user anonymity.

IV. SYSTEM OVERVIEW

In this section, we will introduce the system overview of PPCA, including the architecture, workflow, threat model, and security goal.

A. Entities

In PPCA, it defines three types of entities in the system: 1) provers; 2) issuers; and 3) verifiers [27], which is similar to ACS. These entities behave as follows.

- 1) Provers, who are credential holders, need to be authenticated in order to get access to the system. Each prover holds a set of private identity attributes. Provers wish to protect the privacy of their attributes, so they do not provide their attribute sets in plaintext to others. They aim to disclose only the minimum necessary attributes to the verifier to demonstrate that they meet access requirements.
- 2) Issuers are responsible for signing and issuing credentials for provers. They will receive the requests from provers and respond to them. Although issuers need to interact with provers to generate and issue credentials, they are not trusted by provers.
- 3) Verifiers are responsible for verifying the validity of provers credentials to authenticate them. A verifier holds a requirement set of attributes as a criterion to determine whether to accept the provers. During verification, the verifier needs to check whether the prover meets all the attribute requirements. Provers need to interact with verifiers and disclose part of their attributes. However, provers do not trust verifiers, because verifiers are usually curious and try to get the undisclosed attributes.

B. Workflow

The whole process of PPCA workflow is demonstrated in Fig. 3.

- 1) Prover needs to send the commitment to the attributes to the issuer and request to register.
- 2) The issuer accepts the request and returns a signature on the commitment to the attributes.
- 3) The prover needs to locally generate a rerandomized credential.
- 4) The prover presents the credential and the related witness to the verifier.
- 5) The verifier will authenticate the prover according to the submitted credential and witness. If the verification fails, prover will be rejected.
- 6) If the verification is successful, a session is established between the verifier and the prover. Subsequent requests from the prover to the verifier need to include additional proof to demonstrate that these requests come from the same prover who established the session.

In this article, the process of CA is illustrated in Fig. 4, which shows how the verifier and prover interact during the authentication process.

- 1) Upon receiving credential 1 and credential 2, the verifier performs the legitimacy check to assess the validity of both credentials. If the verifier requires a consistency check to validate the consistency of the sources of the two credentials, it then sends a CA request.
- 2) If the prover receives a CA request from the verifier and decides to send a consistency proof to obtain the verifier's service, the prover will send the proof based

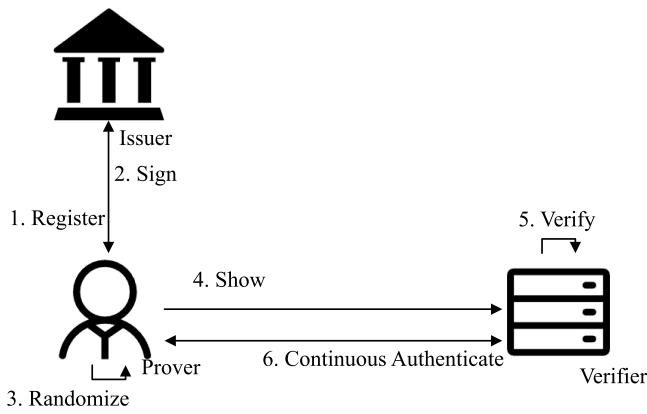


Fig. 3. Workflow of PPCA.

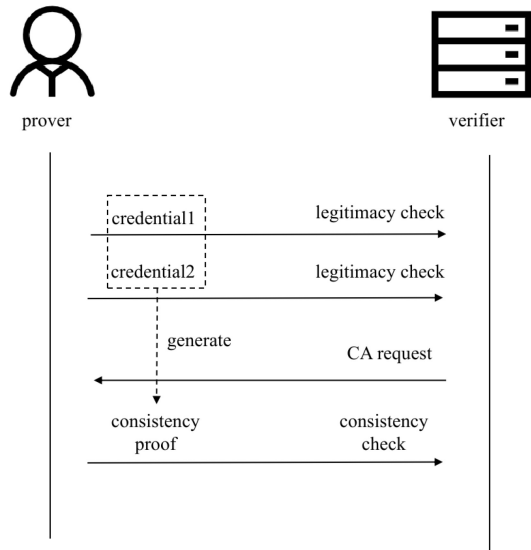


Fig. 4. Process of CA.

on the previously sent credential 1, credential 2, and the CA request. However, if the prover refuses to send the consistency proof, the authentication process is aborted.

- 3) Upon receiving the consistency proof, the verifier validates the identity consistency to ensure that the credentials come from the same anonymous prover.

In our scheme, the prover is not required to provide any private identity information but only needs to supply an optional consistency proof for the consistency check. It prevents the leakage of identity information while ensuring the integrity and consistency of the credentials.

C. Threat Model

In PPCA, each entity in the system can be malicious. In this article, our threat model is as follows.

- 1) *Issuer:* The issuer is honest but curious. They will faithfully execute the protocol steps but will also attempt to obtain the prover's private attributes. Additionally, they may collude with the verifier to reveal the prover's identity without consistency proof.

- 2) *Prover:* The malicious prover aims to bypass the CA of the verifier. Specifically, malicious prover attempts to a) pass the check using a fake credential without having a valid set of attributes and b) impersonate a legitimate user and pass the consistency check without possessing the user's credential, thereby hijacking their identity.
- 3) *Verifier:* The issuer is honest but curious. They will faithfully execute the protocol steps but attempt to a) compromise the private attributes of the prover according to the credential received and b) establish the linkability between credentials without consistency proof to reveal the prover's identity.

D. Security Goal

The proposed PPCA is the combination of ACS and CA to realize SSI and fine-grained CA. Therefore, it has to satisfy the requirements of both of them to realize legitimacy check and consistency check. In this article, we state the security goal of PPCA as follows.

- 1) *Correctness:* Authentication will be accepted if the input provided to the verifier is a valid credential and satisfies the authentication and access control restrictions.
- 2) *Soundness:* A prover cannot feasibly bypass the verification without a valid unaltered credential which was issued by the issuer.
- 3) *Anonymity:* Without the consistency proof provided by the prover, the verifier cannot establish a link between these requests. The verifier cannot determine whether these requests come from the same prover.
- 4) *Linkability:* With the consistency proof provided by the prover, the verifier can continuously authenticate that the requests all come from the same user.

Linkability is the primary difference between PPCA and ACS. In existing ACS systems, it is essential to ensure that the verifier cannot establish linkability when the prover shows credentials multiple times. Such unlinkability ensures the user's privacy while conducting legitimacy checks. However, CA not only requires the verifier to continuously validate the prover's legitimacy but also to verify the consistency of the request source. Therefore, PPCA also needs to provide linkability between multiple credentials to support consistency checks. Moreover, it is required to ensure that the user's privacy is preserved to the greatest extent while realizing linkability.

To achieve linkability, an additional consistency proof is necessary. Each consistency proof can only be used once. It demonstrates the linkability between two credentials and can only be generated by the prover. The prover can selectively provide consistency proof for specific credentials. Furthermore, the consistency proof only demonstrates the relationship between the credentials selected by the prover, without compromising the integrity of the prover's other credentials. As a result, it ensures the preservation of prover privacy in the consistency check process.

In the design of PPCA, it is essential to meet the security requirements of correctness, soundness, anonymity, and

linkability to ensure that PPCA can provide anonymous self-sovereign identity and fine-grained CA.

V. SYSTEM DESIGN

In this section, we will give the definitions of PPCA and the security definition. We will also give a construction of PPCA based on BLS signature, Set commitment, and Sigma Protocol and we will give the security analysis of our construction.

A. Algorithm Definition

PPCA consists of the following algorithms: (*Setup*, *KeyGen*, *Commit*, *Sign*, *CredGen*, *Verify*, *ConGen*, *ConVerify*). *Setup* is to generate the public parameters for the system. *KeyGen* is to generate the keys for entities in the system. *Commit* is for provers to commit to their attributes. *Sign* is for issuers to sign the commitment provided by provers. *CredGen* is for provers to generate credentials locally. *Verify* is for verifiers to check the validity of the credentials provided by provers. *ConGen* is for provers to generate proofs for CA. *ConVerify* is for verifiers to continuously authenticate the prover's identity. We define these algorithms as follows.

Setup(1^λ) $\rightarrow$ *pp*: It is a probabilistic algorithm, which takes as inputs the security parameter λ and outputs the public parameters *pp*.

KeyGen(*pp*) $\rightarrow$ (*sk*, *pk*, *upk*): It is a probabilistic algorithm, which takes as inputs the public parameters *pp* and outputs the secret key *sk*, public key *pk* and update key *upk*.

Commit(*pp*, *S*) $\rightarrow$ *C*: It is a probabilistic algorithm, which takes as inputs the attribute Set of prover *S* and *pp* and outputs the commitment *C* to *S*.

Sign(*pp*, *C*, *sk_i*) $\rightarrow$ σ : It is a deterministic algorithm, which takes as inputs the commitment and issuer's secret key *sk_i* and outputs the signature σ .

CredGen(*pp*, *C*, *S*, *T*, σ) $\rightarrow$ *cred*: It is a probabilistic algorithm, which takes as inputs the public parameters *pp*, the commitments *C*, attribute set *S*, a requirement set *T* and the signature σ and outputs the credential *cred*.

Verify(*pp*, *cred*, *pk_i*) $\rightarrow$ 0/1: It is a deterministic algorithm, which takes as inputs the public parameters *pp*, credential *cred* and public key of the issuer and outputs 1 if *cred* can be accepted and outputs 0 if it cannot be accepted.

The previous algorithm is similar to the formal definition in ACS. However, to apply ACS in CA, we also need to define additional functions, *ConGen* and *ConVerify*, to support consistency checks in CA.

ConVerify(*pp*, *cred₁*, *cred₂*, *pk_i*) $\rightarrow$ 0/1: It is a deterministic algorithm, which takes as inputs the public parameters *pp*, credential *cred*, credential for CA *cred₂* and public key of issuer *pk_i*. If the CA can be accepted by the verifier, it outputs 1, otherwise 0.

B. Security Definition

We have given a brief security goal before. In this section, we will give a formal definition of the security goals.

Correctness: If PPCA is correct, that means for $\forall cred, cred_{Con}$, which is valid and meets the requirements set *T* and *T_{op}*, the following holds:

$$Pr\left(T \subset S, T_{op} \subset S : Verify(pp, cred, pk_i) \wedge ConVerify(pp, C, W)\right) = 1. \quad (7)$$

Soundness: If PPCA is sound, that means for all PPT adversaries $\mathcal{A}$ with oracles $O = (O_{CredGen}, O_{ConGen})$, there is a negligible function *negl* such that

$$Pr\left(cred \leftarrow O_{CredGen}(pp, C, S^*, T, \sigma), cred_{Con} \leftarrow O_{ConGen}(pp, C, S^*, T_{op}, \sigma) : Verify(pp, cred, pk_i) \wedge ConVerify(pp, C, W)\right) = 1 \leq negl(\kappa). \quad (8)$$

Anonymity: For credentials without consistency proof, the verifier cannot distinguish the owner of the credentials. Suppose there are two sets *S₀*, *S₁*, the verifier cannot distinguish whether *cred₀* and *cred₁* come from the same set without the proof

$$Pr\left(a, b \leftarrow_{\$} \{0, 1\}, cred_0 \leftarrow CredGen(pp, C_0, S_a, T, \sigma) cred_1 \leftarrow CredGen(pp, C_1, S_b, T, \sigma) : a \stackrel{?}{=} b \leftarrow \mathcal{A}(cred_0, cred_1)\right) - \frac{1}{2} \leq negl(\kappa). \quad (9)$$

Linkability: For credentials with a consistency proof, the verifier can link them to the same prover who creates them. Suppose *cred_{con}* is provided by the prover

$$Pr\left(a, b \leftarrow_{\$} \{0, 1\}, cred_0 \leftarrow CredGen(pp, C_0, S_a, T, \sigma) cred_1 \leftarrow ConGen(pp, C_1, S_b, T, \sigma) : ConVerify(pp, cred_0, cred_1, pk_i) = 1\right) = 1. \quad (10)$$

Linkability does not compromise anonymity. This is because the verifier can only determine whether the credentials come from the same prover, but cannot determine which prover they come from.

C. Construction

In this section, we will give an instance of PPCA. The specific process is described as follows.

Assume the prover possesses a set of attributes *S*, and the verifier holds a requirement set *T*. The user aims to send a credential to the verifier, proving that it meets all access requirements $T \subset S$. Meanwhile, the prover wishes to keep its set *S* private, while the verifier's set *T* remains public. Additionally, it is necessary to satisfy a consistency check to achieve CA.

Setup(1^λ) $\rightarrow$ *pp*: It takes as input the security parameter λ . Suppose that the length of the attributes set is *n*. It chooses a bilinear group $BG = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$. It also needs to conduct *SC.Setup* for set commitment. It randomly chooses $\alpha \leftarrow_{\$} \mathbb{Z}_p$, and generates the reference string $(g_1^{\alpha^i}, g_2^{\alpha^i})_{i \in [n]}$. It randomly chooses $a \leftarrow_{\$} \mathbb{Z}_p$ and computes

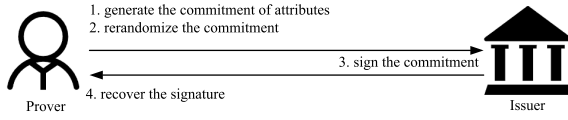


Fig. 5. Confidential BLS signature for the commitment.

$h = g_1^a, (h^{\alpha^i})_{i \in [n]}$. It then returns the public parameters: $pp = (BG, g_1^{\alpha^i}, g_2^{\alpha^i}, h, h^{\alpha^i})$. α and a will be dropped after setup.

$KeyGen(pp) \rightarrow (sk, pk, upk)$: It takes as input the public parameters pp . The issuer can randomly choose a secret key $sk = \beta \leftarrow \mathbb{Z}_p$ and generate the public key $pk = (g_1^\beta, g_2^\beta)$. The secret key is for issuer to sign a signature and the public key is for verifiers to verify the validity of credentials from provers. It will also generate an update key $upk = h^\beta$ for provers to update their signatures locally for multishow to keep privacy-preserving.

$Commit(pp, S) \rightarrow C$: It takes as inputs the attribute Set of prover $S = \{s_i\}_{i \in n}$ and pp . It follows the steps in SC.Commit. It randomly chooses $\rho \leftarrow \mathbb{Z}_p$, encodes the set into a polynomial $f_s(X) = \prod_{s \in S} (X - s) = \sum_{i \in [n]} f_i X^i$ and commits to it $C = (\prod_{i \in [S]} g_1^{\alpha^i f_i})^\rho, \rho \leftarrow \mathbb{Z}_p$. It finally outputs the commitment C to S .

$Sign(pp, C, pk_i, sk_i) \rightarrow \sigma$: It takes as inputs the commitment and issuer secret key sk_i and outputs the signature σ . However, the prover does not want to expose the commitment to issuers directly if the issuer uses the prover commitment to misuse its identity.

Therefore, the prover needs to randomize the commitment before sending it

$$\hat{C} = g_1^y C, y \leftarrow \mathbb{Z}_p \quad (11)$$

where $\hat{C}$ is the randomized commitment and y is a random number generated by prover. Then, the prover will send $\hat{C}$ instead of C to the issuer.

The issuer will sign the commitment by running $\hat{\sigma} = BLS.sign(\hat{C}, sk_i) = \hat{C}^\beta$ and return it to the prover. In this article, we use Set Commitment to work as the hash function $H: \{0, 1\}^* \rightarrow G_1$ mentioned in the BLS signature, which can map the attribute set into a group element.

Finally, the prover will recover the valid signature with the public key of issuer $pk = (g_1^\beta, g_2^\beta)$, $\sigma = \hat{\sigma} (g_1^\beta)^{-y} = C^\beta$.

The whole process can be considered as a simple version of confidential BLS signature as shown in Fig. 5. The issuer cannot get the real commitment C while the prover can recover the valid signature σ to the commitment. We will discuss the correctness later.

$CredGen(pp, C, S, T, \sigma) \rightarrow cred$: It takes as inputs the public parameters pp , the commitments C , and the signature σ . In this step, the prover needs to rerandomize the credentials, to make the credentials different in each show. It needs to sample a randomness $z \leftarrow \mathbb{Z}_p$ and rerandomize the commitment and signature according to the randomness z , $\tilde{C} = Ch^z, \tilde{\sigma} = \sigma (h^\beta)^z$. To convince the verifier that the prover holds a valid credential that meets the requirements set T from the verifier. It also needs to generate a witness $W = SC.OpenSubset(pp, C, S, T) = C_{S/T} = (\prod_{i \in [n-|T|]} g_1^{\alpha^i f_i})^\rho$, and

then rerandomize the witness $\tilde{W} = Wh^z$. The rerandomization is different from the existing ACS based on Set Commitment to support the construction of consistency proof. Then, it needs to compute g_2^z . Finally, the prover will output the credential

$$cred = (\tilde{C} = Ch^z, \tilde{W} = Wh^z, \tilde{\sigma} = \sigma (h^\beta)^z, g_2^z). \quad (12)$$

$Verify(pp, cred, pk_i) \rightarrow 0/1$: It takes as inputs the public parameters pp , credential $cred$ and public key of issuer pk_i . To check the validity of the credential, the verifier needs to run the following check:

$$e(\tilde{C}, g_2^\beta) = e(\tilde{\sigma}, g_2) \wedge SC.Verify(\tilde{C}, \tilde{W}, T) = 1. \quad (13)$$

Since the rerandomization of our construction is different from the existing methods, $SC.Verify(\tilde{C}, \tilde{W}, T)$ is to check that

$$\frac{e(\tilde{W}, \prod_{i \in [T]} g_2^{\alpha^i f_i})}{e(\prod_{i \in [T]} h^{\alpha^i f_i}, g_2^z)} \stackrel{?}{=} \frac{e(\tilde{C}, g_2)}{e(h, g_2^z)}. \quad (14)$$

$e(\tilde{C}, g_2^\beta) = e(\tilde{\sigma}, g_2)$ proves that the rerandomized commitment is valid, and generated from a credential signed by the issuer. $SC.Verify(\tilde{C}, \tilde{W}, T) = 1$ proves that the credential satisfies the requirements set T from the verifier. It will output 1 if both of the pairings can be accepted and outputs 0 otherwise.

$ConVerify(pp, cred_1, cred_2, pk_i) \rightarrow 0/1$: It takes as inputs the public parameters pp , credential $cred_1$, credential for CA $cred_2$ and public key of issuer pk_i . If the CA can be accepted by the verifier, it outputs 1, otherwise 0.

When the prover is accepted by the issuer according to the result of $Verify$, a session is established. However, according to the security requirements of zero-trust architecture networks, CA rather than one-time authentication is recommended. CA continuously monitors and verifies the user's identity throughout the session to ensure it remains trustworthy. Therefore, when a prover requests for subsequent operation op , both the consistency check and legitimacy check need to be conducted.

The CA can be divided into two steps.

In step 1, the prover needs to prove the identity consistency by a Sigma protocol to convince the verifier that it is the same prover of the two credentials. As shown in Fig. 6, suppose the prover has sent two commitments $\tilde{C}_1 = Ch^z, \tilde{C}_2 = Ch^t, z, t \leftarrow \mathbb{Z}_p$ to the verifier. The prover needs to prove that $\tilde{C}_1$ and $\tilde{C}_2$ are generated from the same C without revealing C, z, t , and the polynomial $f_s(X)$.

1) Prover samples $r_e, r_z, r_t \leftarrow \mathbb{Z}_p$.

2) Prover sends $\theta_1 = g_1^{\sum_{i \in n} \alpha^i r_e} h^{r_z}, \theta_2 = g_2^{\sum_{i \in n} \alpha^i r_e} h^{r_t}$.

3) Verifier returns a challenge c .

4) Prover computes $s_{e_i} = r_e - c f_i \rho, s_z = r_z - c z, s_t = r_t - c t$ and sends them to verifier.

5) Verifier checks that $\theta_1 \stackrel{?}{=} \tilde{C}_1^c g_1^{\sum_{i \in n} \alpha^i s_{e_i}} h^{s_z}, \theta_2 \stackrel{?}{=} \tilde{C}_2^c g_1^{\sum_{i \in n} \alpha^i s_{e_i}} h^{s_t}$.

By the proof process above, the prover can demonstrate the consistency between $\tilde{C}_1$ and $\tilde{C}_2$ without revealing any additional information about the attribute set. We will discuss the correctness of this proof in the next section. Besides, The above interactive Sigma protocol could be converted

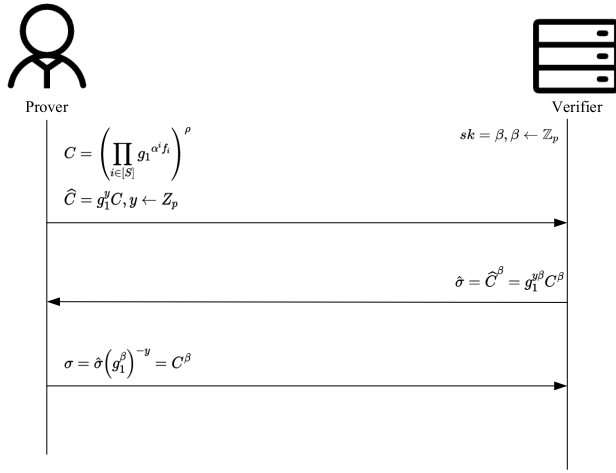


Fig. 6. Sigma protocol for consistency proof.

into noninteractive via the Fiat–Shamir transformation [40] to reduce the communication cost.

In step 2, the prover needs to prove the operation authorization to convince the verifier that the credential provided meets the requirements set T_{op} for operation op . This step is similar to the identity authentication, the only difference is that the requirements set is T_{op} according to the operation op the prover requests. The prover needs to generate the witness $\text{cred}_{\text{Con}} = \text{CredGen}(pp, C, S, T_{op}, \sigma)$, where the randomness is $\kappa \leftarrow_{\$} \mathbb{Z}_p$. The verifier will verify the validity by running $\text{Verify}(pp, \text{cred}_2, pk_i)$. If accepted, it will return 1, and 0 otherwise.

D. Security Analysis

In this section, we will discuss the security performance of PPCA. First, we need to introduce two assumptions.

Decisional Diffie–Hellman Assumption (DDH): Suppose that $BG = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$ and there is a sequence $(g_1^{a_i}, g_2^{a_i})$. Then the DDH holds if for all probabilistic polynomial-time adversaries $\mathcal{A}$, there is a negligible function $\text{negl}(\kappa)$, for randomness $r, s, t \in \mathbb{Z}_p$ such that

$$\begin{aligned} & \Pr(b^* \xleftarrow{R} \mathcal{A}(BG, (g_1^{a_i}, g_2^{a_i})_{i \in [q]} \\ & \quad g_1^r, g_1^s, ((1-b) \cdot t + b \cdot rs)) : \\ & \quad b^* = b) - \frac{1}{2} \leq \text{negl}(\kappa). \end{aligned} \quad (15)$$

q-co-Generalized Strong Diffie–Hellman Assumption (q-co-GSDH): Suppose that $BG = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$ and there is a sequence $(g_1^{a_i}, g_2^{a_i})$. Then the q-co-GSDH holds if for all probabilistic polynomial-time adversaries $\mathcal{A}$, there is a negligible function $\text{negl}(\kappa)$, such that

$$\begin{aligned} & \Pr((g, h, T) \xleftarrow{R} \mathcal{A}(BG, (g_1^{a_i}, g_2^{a_i})_{i \in [q]})) : \\ & \quad T \in \mathbb{G}_1 \wedge g, h \in \mathbb{Z}_p[X] \wedge \\ & \quad 0 \leq \deg g < \deg h \leq q \wedge \\ & \quad e(T, h(a)\hat{P}) = e(g(a)P, \hat{P}) \leq \text{negl}(\kappa). \end{aligned} \quad (16)$$

Next, we will discuss the security goals of PPCA.

Correctness: Consider the attribute set of prover $S = \{s_i\}$. After running $\text{Commit}(pp, S)$, it get a commitment $C = (\prod_{i \in [S]} g_1^{\alpha^i f_i})^\rho, \rho \leftarrow_{\$} \mathbb{Z}_p$. After rerandomization, the commitment turns to $\hat{C} = g_1^\beta C, y \leftarrow_{\$} \mathbb{Z}_p$ which is provided to the issuer to generate a signature $\hat{\sigma} = \hat{C}^\beta$. Therefore, the prover can recover the signature as follows:

$$\hat{\sigma} (g_1^\beta)^{-y} = \hat{C}^\beta (g_1^\beta)^{-y} = g_1^{y\beta} C^\beta (g_1^\beta)^{-y} = C^\beta = \sigma. \quad (17)$$

Then, the prover can generate the credential locally according to the recovered signature σ , $\text{cred} = (\tilde{C} = Ch^z, \tilde{W} = Wh^z, \tilde{\sigma} = \sigma(h^\beta)^z, g_2^z, W = C_{S/T} = (\prod_{i \in [n-|T|]} g_1^{\alpha^i f_i})^\rho$. Finally, the verifier will check the validity of the signature and commitment.

First, it will check the validity of the signature

$$e(\tilde{C}, g_2^\beta) = e(Ch^z, g_2^\beta) = e(C^\beta (h^z)^\beta, g_2) = e(\tilde{\sigma}, g_2). \quad (18)$$

Then, it will check the validity of the commitment. Since

$$\begin{aligned} & \frac{e(\tilde{W}, \prod_{i \in [|T|]} g_2^{\alpha^i f_i})}{e(\prod_{i \in [|T|]} h^{\alpha^i f_i}, g_2^z)} \\ &= \frac{e(W, \prod_{i \in [|T|]} g_2^{\alpha^i f_i}) e(h^z, \prod_{i \in [|T|]} g_2^{\alpha^i f_i})}{e(h^z, \prod_{i \in [|T|]} g_2^{\alpha^i f_i})} \\ &= e\left(W, \prod_{i \in [|T|]} g_2^{\alpha^i f_i}\right) \end{aligned} \quad (19)$$

$$\frac{e(\tilde{C}, g_2)}{e(h, g_2^z)} = \frac{e(C, g_2) e(h, g_2^z)}{e(h, g_2^z)} = e(C, g_2) \quad (20)$$

$$e\left(W, \prod_{i \in [|T|]} g_2^{\alpha^i f_i}\right) = e(C, g_2). \quad (21)$$

Therefore

$$\frac{e(\tilde{W}, \prod_{i \in [|T|]} g_2^{\alpha^i f_i})}{e(\prod_{i \in [|T|]} h^{\alpha^i f_i}, g_2^z)} = \frac{e(\tilde{C}, g_2)}{e(h, g_2^z)}. \quad (22)$$

That is to say, for $T \subset S$, $\text{Verify}(pp, \text{cred}, pk_i) = 1$. Therefore, we meet the correctness in Section V-B for honestly computed and aggregated signatures.

Soundness: Suppose an adversary $\mathcal{A}$ can generate $\text{cred} = \text{CredGen}(pp, C, S^*, T, \sigma)$, where $T \not\subset S^*$. It satisfies $\text{Verify}(pp, \text{cred}, pk_i) = 1$. Then, we can construct an adversary $\mathcal{B}$ against q-co-GSDH assumption, according to [41].

$\mathcal{B}$ first needs to generate a commitment for set S^*

$$C_{S^*} = \left(\prod_{i \in [S^*]} g_1^{\alpha^i f_i}\right)^\rho. \quad (23)$$

Since adversary $\mathcal{A}$ satisfies $\text{Verify}(pp, \text{cred}, pk_i) = 1$, so $\text{SC.Verify}(C_{S^*}, W, T) = 1$

$$e\left(W, \prod_{i \in [|T|]} g_2^{\alpha^i f_i}\right) = e(C_{S^*}, g_2). \quad (24)$$

Therefore

$$e(W^{\rho^{-1}}, g_2^{f_T(\alpha)}) = e(g_1^{f_{S^*}(\alpha)}, g_2). \quad (25)$$

In the scenario discussed in this article, $|S^*| \geq |T|$. This is because the number of attributes held by the adversary (prover) exceeds the number of attributes required by the verifier. Therefore, the construction can be done through polynomial decomposition as follows: $f_{S^*}(x) = h(x)f_T(x) + r(x)$, where $\deg(S^*) \geq \deg(T)$. Then

$$e(W^{\rho^{-1}}, g_2^{f_T(\alpha)}) = e(g_1^{h(\alpha)f_T(\alpha)+r(\alpha)}, g_2) \quad (26)$$

$$e\left(\frac{W^{\rho^{-1}}}{g_1^{h(\alpha)}}, g_2^{f_T(\alpha)}\right) = e(g_1^{r(\alpha)}, g_2) \quad (27)$$

$(r(\alpha), f_T(\alpha), [W^{\rho^{-1}}/g_1^{h(\alpha)}])$ is a solution to the q-co-GSDH problem.

Meanwhile, the prover cannot gain an advantage by modifying the signature σ . This is because the private key sk_i for signing is securely stored by the Issuer. The prover can only recover a valid signature by running Sign but cannot obtain the signing private key or generate signatures for additional attributes.

Therefore, we meet the soundness in Section V-B.

Anonymity: Suppose that there is an adversary $\mathcal{A}$ that can distinguish the $cred_0$ and $cred_1$ from two sets without the consistency proof, then we can construct an adversary $\mathcal{B}$ against the DDH assumption.

$\mathcal{A}$ can get the witness from credential that

$$W = C_{S/T} = g_1^{f_{S/T}(\alpha)\rho}. \quad (28)$$

And it can generate the commitment for set T

$$C_T = g_1^{f_T(\alpha)\rho}. \quad (29)$$

Then, we can construct the following:

$$I = \left(g_1^{f_T(\alpha)f_{S/T}(\alpha)}\right)^\rho h^{(1-b)z_0} \left(g_1^{f_S(\alpha)}\right)^\rho h^{bz_1} \quad (30)$$

where $b \leftarrow \{0, 1\}$. When $b = 0$, $I = Ch^{z_0} = \tilde{C}_0$. When $b = 1$, $I = Ch^{z_1} = \tilde{C}_1$. Therefore, if $\mathcal{A}$ can distinguish the $cred_0$ and $cred_1$, which means it can distinguish $\tilde{C}_0$ and $\tilde{C}_1$, then we can get the result of I . (W, C_T, C, I) is a solution to DDH problem.

For the verifier, it is impossible to establish linkability between two credentials without consistency proof, determining whether they originate from the same prover. This is because, through credential rerandomization, credentials from the same prover and those from different provers become indistinguishable. From the user's perspective, these credentials are indistinguishable from newly generated credentials. Therefore, we meet the anonymity in Section V-B.

Linkability: With the consistency proof, the verifier can establish the linkability of $cred_1$ and $cred_2$, by checking the relationship of $\theta_1, \theta_2, c, s_{ei}, s_z, s_t$. If the prover is honest, then

$$\begin{aligned} \tilde{C}_{18}^c g_1^{\sum_{i \in n} \alpha^i s_{ei}} h^{s_z} &= C^c h^{cz} g_1^{\sum_{i \in n} \alpha^i s_{ei}} h^{s_z} \\ &= g_1^{\sum_{i \in n} \alpha^i f_{icp}} h^{cz} g_1^{\sum_{i \in n} \alpha^i s_{ei}} h^{s_z} \end{aligned}$$

TABLE I
FUNCTIONALITY COMPARISON BETWEEN PPCA AND THE SCHEMES
PRESENTED IN [12], [15], [42], AND [43]

	Soundness	Anonymity	Linkability	Privacy
[15]	✓	✓	✓	×
[12]	✓	✓	forced	×
[42]	✓	✓	forced	×
[43]	✓	✓	TTP needed	×
PPCA	✓	✓	✓	✓

$$\begin{aligned} &= g_1^{\sum_{i \in n} \alpha^i r_e} h^{r_z} \\ &= \theta_1 \end{aligned} \quad (31)$$

$$\begin{aligned} \tilde{C}_2^c g_1^{\sum_{i \in n} \alpha^i s_{ei}} h^{s_t} &= C^c h^{ct} g_1^{\sum_{i \in n} \alpha^i s_{ei}} h^{s_t} \\ &= g_1^{\sum_{i \in n} \alpha^i f_{icp}} h^{cz} g_1^{\sum_{i \in n} \alpha^i s_{ei}} h^{s_t} \\ &= g_1^{\sum_{i \in n} \alpha^i r_e} h^{r_t} \\ &= \theta_2. \end{aligned} \quad (32)$$

After that, the prover also need to prove that $cred_2$ meet the access requirement of verifier by $Verify(pp, cred_2, pk_i)$

With consistency proof, each time the prover requests an operation op , it must provide additional proof. The prover needs to prove that: 1) the request comes from the prover who established the session connection and 2) the prover meets the access control requirements T_{op} for accessing that operation. Therefore, with the proof provided by the prover, the verifier can always establish a linkability between $cred_1$ and $cred_2$. Therefore, we meet the linkability in Section V-B.

Table I compares the function of PPCA and the existing schemes proposed in [12], [15], [42], and [43]. The table takes soundness, anonymity, linkability, and privacy into consideration. The existing schemes are based on the LRS or the linkable group signature, which can support soundness and anonymity. However, they require the signed message to be disclosed to the verifier. These signed messages contain users' private identity information in CA, which means that the privacy performance of these schemes is not good enough. In [12] and [42], the schemes they proposed allow the signature generated by the user to be unconditionally checked by the verifier, which fails to realize the user-centric consistency check. In [43], the scheme needs an extra trusted third party to check the linkability of two messages. Compared to these methods, PPCA protects users' privacy information from disclosure through confidential signatures and set commitments. Moreover, PPCA allows users to independently decide (rather than being compelled) to generate consistency proof for the consistency check, without the need for a trusted third party.

VI. EVALUATION

In this section, we will present an evaluation of the proposed construction PPCA. Our implementation is based on Omid's work [37]. We employ the set commitment and ZKP from their repository to construct PPCA, implementing legitimacy checks and consistency checks.

TABLE II
RUNNING TIME FOR PPCA WHEN $n = 25$ (MS)

	<i>Setup</i>	<i>KeyGen</i>	<i>Commit</i>	<i>Sign</i>	<i>CredGen</i>	<i>Verify</i>	<i>ConGen</i>	<i>ConVerify</i>
average time	307	16	28	75	36	33	35	18

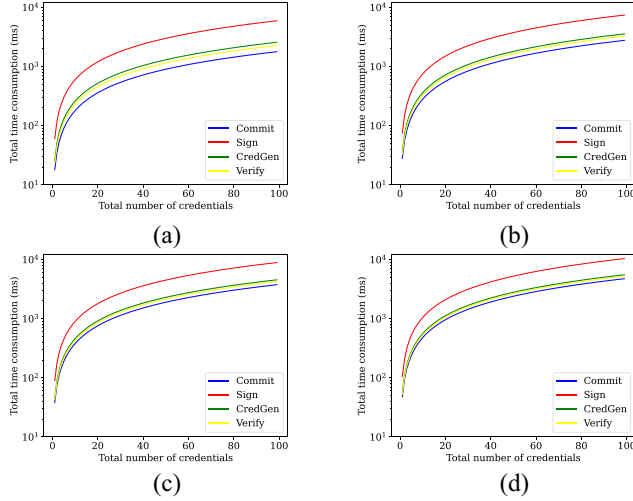


Fig. 7. Running time for PPCA without consistency proof and with different n . (a) $n = 20$. (b) $n = 25$. (c) $n = 30$. (d) $n = 35$.

The implementation follows the construction in Section V. Our experiment is conducted on an Intel Core i5-1155G7 CPU, 16-GB RAM running Ubuntu 22.04.1. For clarity in our discussion, we define the following parameters in our experiments: n represents the size of the attribute set held by each prover in the system, m represents the number of credentials sent from the prover to the verifier without consistency proof, and m' represents the number of credentials sent from the prover to the verifier with consistency proof. The value of n depends on the application scenario. Increasing n leads to higher performance overhead. In this article, we focus on investigating the impact of n on the performance of PPCA, and therefore, we set n from 20 to 35, to observe its effect on efficiency. The value of m varies according to the security requirements. High-security systems, such as those in banking or government, may require authentication once per minute. To demonstrate that PPCA can meet the demands of authentication frequencies, we set m from 1 to 100.

First, we test the running time of each function in the PPCA construction proposed in this article. We conduct a total of 100 rounds of tests and take the average time. The results are presented in Table II.

We are particularly concerned with the time efficiency of each function under varying sizes of attribute sets and different rounds of interactions between the prover and verifier. Since the *Setup* and *KeyGen* are relatively independent components of the system, we will primarily focus on the performance of the other functions.

As shown in Fig. 7, when the total number of credentials increases from $m = 1$ to $m = 100$, the overall time cost of

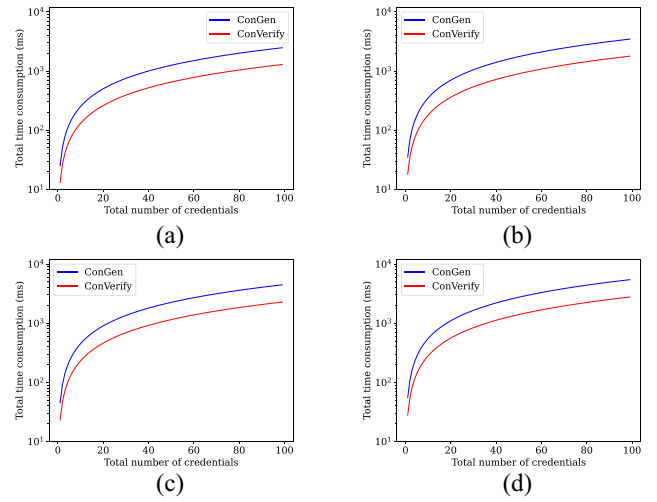


Fig. 8. Running time for PPCA with consistency proof and different n . (a) $n = 20$. (b) $n = 25$. (c) $n = 30$. (d) $n = 35$.

each function increases correspondingly. However, the total time consumed for 100 executions of a single function still does not exceed 10 s, making the time cost overall acceptable. Additionally, a horizontal comparison across different attribute sizes, from $n = 20$ to $n = 35$, reveals that the impact of attribute size on time cost is almost negligible.

Next, we need to test the performance of each function with consistency proof. In this case, the prover and verifier need to additionally execute *ConGen* and *ConVerify*, while the rest of the operations are similar to the operations without sending a consistency proof, our test will focus on the performance of *ConGen* and *ConVerify*.

As shown in Fig. 8, when the total number of credentials increases from $m' = 1$ to $m' = 100$, the overall time cost of *ConGen* and *ConVerify* increases. Additionally, a horizontal comparison across different attribute sizes, from $n = 20$ to $n = 35$, reveals that the impact of attribute size on time cost is almost negligible.

A comparison between Figs. 7 and 8 reveals that the delay introduced by *ConGen* and *ConVerify* is approximately 10% of other functions. This indicates that the delay introduced by *ConGen* and *ConVerify* has a limited impact on the overall system. Moreover, this delay primarily arises to satisfy the fine-grained CA in PPCA, which is therefore considered acceptable.

In addition to time consumption, we also take the communication overhead under consideration. Since CA requires frequent authentication, smaller credential sizes can help reduce the communication overhead, improving payload efficiency and ensuring communication quality. We compared the communication cost of PPCA with [15] and [43]. In Table III, the theoretical size of credentials in PPCA and the signature sizes from [43] and [15] are presented. To provide a clearer comparison, we have analyzed these sizes on the BN256 curve. As shown in Fig. 9, PPCA has a smaller credential size. It indicates that PPCA does not significantly affect communication quality, as the reduced credential size minimizes the communication overhead.

TABLE III
THEORY SIZE OF CREDENTIAL AND SIGNATURE

	[43]	[15]	PPCA
size	$3 \mathbb{G}_1 + 7 \mathbb{Z}_p $	$4 \mathbb{G}_1 + 5 \mathbb{Z}_p $	$3 \mathbb{G}_1 + \mathbb{G}_2 $

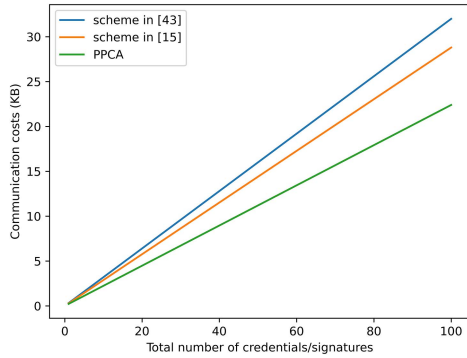


Fig. 9. Communication costs of PPCA and [15], [43].

VII. DISCUSSION

This work tentatively explores a solution that can provide privacy-preservation and CA simultaneously, while there are still some issues in PPCA that need to be further considered. In this section, we discuss PPCA from the following four aspects and give some suggestions for future work.

A. Decision Model

In PPCA, users are suggested to decide by themselves whether to send consistency proof to establish the linkability. Accordingly, a critical issue is whether the users have the background knowledge to make a clear decision in practical applications. Although PPCA allows the user (prover) to decide whether to send a consistency proof, users may be confused during decision-making. It is recommended that the users consider the following factors when determining whether to provide consistency proof to the verifier: transparency, security requirements, and potential risks.

The answer is positive for some simple cases. For example, when users engage in prolonged sessions with anonymous Internet platforms, such as anonymous discussion boards or anonymous social apps, they would prefer to send consistency proof to establish linkability among their requests. In such scenarios, users need to protect their identity privacy to avoid exposing or even compromising their true identity; meanwhile, to ensure the usability of the application, it is necessary to maintain the consistency of the user's interactions over time. Conversely, if users lack trust in the website service provider they are accessing and only wish to establish a one-time request, they would not send consistency proof to maintain their privacy and anonymity to the maximum extent.

Nevertheless, the decision is not that easy to make for a more complicated scenario. In such situations, a decision model should be built to help users decide whether they should establish the linkability more rationally. The model should consider multiple factors, such as the trustworthiness of the

target verifier, the confidentiality of user identity information, and the necessity of establishing consistent verification.

B. Rerandomized Public Keys

In PPCA, users can randomize the credentials to eliminate the linkability between the credentials, which can ensure the anonymity of users. Although PPCA protects user privacy and anonymity in the content of credentials, curious verifiers may still find other ways to establish linkability between user credentials through side channels. For instance, curious verifiers may utilize the user's public key to establish the linkability of credentials in different sessions, without regard to the content of the credential itself [44]. In this case, curious verifiers can establish an index linking credentials to user identities using the user's public key as the primary key, threatening the anonymity of the user's identity.

To deal with the problem, PPCA can be integrated with techniques, such as rerandomized public keys, to further enhance user anonymity in the future. Rerandomized public keys can update the private-public keys of users, avoiding the curious verifiers to establish the linkability of credentials according to users' public keys [45], [46], [47]. Such a technique will allow the user to generate different public keys for each time and the corresponding credentials change accordingly.

C. Remove Trusted Setup

Since our construction of PPCA introduces ACS into CA, PPCA does not need a trusted third party during the operation to realize the legitimate check and consistency check, which follows the requirements of zero-trust architecture networks. However, this scheme still needs a trusted setup. Let's review the construction of *Setup*: $pp = (BG, g_1^{\alpha}, g_2^{\alpha})$, $\alpha \leftarrow \mathbb{Z}_p$. α is sampled by a trusted third party and deleted after the setup. The trusted setup can be achieved by introducing a one-time trusted third party into the PPCA protocol. The third party is only used once during the setup phase and is subsequently discarded, which has been applied in recent works that require a trusted setup [48], [49]. Nevertheless, it is still challenging to find such a trusted third party, and we can hardly ensure whether they have backups. Therefore, we need to implement a construction in the future that can remove the trusted setup.

In this article, the trusted setup is intended to support the zero-knowledge proof based on Set Commitments. It is suggested to explore zero-knowledge proof schemes that do not require a trusted setup as an alternative building block to avoid the problem. For instance, Spartan [50], Bulletproof [51], or Aurora [52] could serve as alternatives. In the future, these transparent setup zero-knowledge proof schemes can be used as alternative building blocks and construct PPCA without a trusted setup.

D. Revocable Credential

In the construction of PPCA proposed in this article, a large number of randomized temporary credentials are generated and users are responsible for revoking these temporary credentials. But in practice, users might leak the temporary

credentials due to insecure revocation. For example, credentials may be carelessly discarded or fail to be revoked promptly. The leakage of these credentials could be utilized by malicious attackers to impersonate the prover's identity. Curiously verifiers could also establish the credentials linkability based on these discarded credentials, revealing the prover's true identity.

Therefore, it is also suggested to introduce revocation to promptly deal with these temporary credentials. Recently, there have been many kinds of research about revocable ACS [32], [53]. These schemes may use accumulators or merkle trees to support revocation. By introducing the revocable ACS into CA scenario, a revocable PPCA can be constructed, and users can have a better choice to deal with their temporary credentials in the future.

E. Other Possible Solutions

The key cause of privacy leakage in CA is that users submit their identity information to the verifier without protection. This is similar to privacy protection challenges in cloud services [54], [55], [56] and blockchain [57], [58]. Researchers have novelly applied machine learning [54], [56] to ensure sensitive data is not leaked to untrusted third parties and explored private data assignment on such platforms [55], [57]. While existing research has not specifically addressed CA, related studies provide potential solutions. Future work should focus on designing protocols that balance secure authentication and privacy protection.

VIII. CONCLUSION

In this article, we propose anonymous credential authentication. It enables verifiers to continually authenticate users' identities while preserving users' privacy. It also ensures the consistency check of user identity and facilitates fine-grained CA. We propose a formal definition of the PPCA along with its security goals. Additionally, we present a construction of the PPCA based on BLS signatures, Set Commitment, and Sigma Protocol. PPCA has an advantage in scenarios where it is necessary to prove user identity consistency while maintaining the anonymity of the user. Through security analysis and evaluation, we demonstrate that PPCA can meet security requirements with limited efficiency loss. Finally, we discuss the remaining open challenges of PPCA and potential future research directions.

REFERENCES

- [1] C. Zhao, F. Gao, and Z. Shen, "Multi-motion sensor behavior based continuous authentication on smartphones using gated two-tower transformer fusion networks," *Comput. Security*, vol. 139, Apr. 2024, Art. no. 103698. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404823006089>
- [2] P. Aaby, M. V. Giuffrida, W. J. Buchanan, and Z. Tan, "An omnidirectional approach to touch-based continuous authentication," *Comput. Security*, vol. 128, May 2023, Art. no. 1031463. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404823000561>
- [3] I. Stylios, S. Chatzis, O. Thanou, and S. Kokolakis, "Continuous authentication with feature-level fusion of touch gestures and keystroke dynamics to solve security and usability issues," *Comput. Security*, vol. 132, Sep. 2023, Art. no. 103363. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404823002730>

- [4] H. Alamleh and A. A. S. AlQahtani, "Architecture for continuous authentication in location-based services," in *Proc. Int. Conf. Innov. Intell. Inform., Comput. Technol. (3ICT)*, 2020, pp. 1–4.
- [5] T. G. Calderon, A. Chandra, and J. J. Cheh, "Modeling an intelligent continuous authentication system to protect financial information resources," *Int. J. Account. Inf. Syst.*, vol. 7, no. 2, pp. 91–109, 2006.
- [6] A. K. Sahu, S. Sharma, and R. Raja, "Deep learning-based continuous authentication for an IoT-enabled healthcare service," *Comput. Electr. Eng.*, vol. 99, Apr. 2022, Art. no. 107817. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S004579062200115X>
- [7] S. Almalki, N. Assery, and K. Roy, "An empirical evaluation of online continuous authentication and anomaly detection using mouse clickstream data analysis," *Appl. Sci.*, vol. 11, no. 13, p. 6083, 2021.
- [8] V. Mavroudis, "Zero-trust network access (ZTNA)," 2024, *arXiv:2410.20611*.
- [9] L. Meng, D. Huang, J. An, X. Zhou, and F. Lin, "A continuous authentication protocol without trust authority for zero trust architecture," *China Commun.*, vol. 19, no. 8, pp. 198–213, Aug. 2022.
- [10] A. Acar et al., "A lightweight privacy-aware continuous authentication protocol-PACA," *ACM Trans. Privacy Security*, vol. 24, no. 4, pp. 1–28, Sep. 2021. [Online]. Available: <https://dl.acm.org/doi/10.1145/3464690>
- [11] A. F. Baig and S. Eskeland, "Security, privacy, and usability in continuous authentication: A survey," *Sensors*, vol. 21, no. 17, p. 5967, 2021.
- [12] Y. Liu, D. He, Z. Bao, H. Wang, M. K. Khan, and K.-K. R. Choo, "An efficient multilayered linkable ring signature scheme with logarithmic size for anonymous payment in vehicle-to-grid networks," *IEEE Trans. Intell. Veh.*, vol. 8, no. 4, pp. 2998–3011, Apr. 2023.
- [13] J. Odoom, X. Huang, Z. Zhou, S. Danso, J. Zheng, and Y. Xiang, "Linked or unlinked: A systematic review of linkable ring signature schemes," *J. Syst. Archit.*, vol. 134, Jan. 2023, Art. no. 102786. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1383762122002715>
- [14] M. N. S. Perera, T. Nakamura, M. Hashimoto, H. Yokoyama, C.-M. Cheng, and K. Sakurai, "A survey on group signatures and ring signatures: Traceability vs. anonymity," *Cryptography*, vol. 6, no. 1, p. 3, 2022.
- [15] J. Diaz and A. Lehmann, "Group signatures with user-controlled and sequential linkability," in *Proc. IACR Int. Conf. Public-Key Cryptogr.*, 2021, pp. 360–388.
- [16] J. Y. Hwang, S. Lee, B.-H. Chung, H. S. Cho, and D. Nyang, "Short group signatures with controllable linkability," in *Proc. Workshop Lightweight Security Privacy, Devices, Protocols, Appl.*, 2011, pp. 44–52.
- [17] H. Feng, J. Liu, D. Li, Y.-N. Li, and Q. Wu, "Traceable ring signatures: General framework and post-quantum security," *Designs, Codes Cryptogr.*, vol. 89, pp. 1111–1145, Mar. 2021.
- [18] F. H. Al-Naji and R. Zagrouba, "A survey on continuous authentication methods in Internet of Things environment," *Comput. Commun.*, vol. 163, pp. 109–133, Nov. 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0140366420319204>
- [19] R. Rocha, D. Carneiro, and P. Novais, "Continuous authentication with a focus on explainability," *Neurocomputing*, vol. 423, pp. 697–702, Jan. 2021.
- [20] I. C. Stylios, O. Thanou, I. Androulidakis, and E. Zaitseva, "A review of continuous authentication using behavioral biometrics," in *Proc. SouthEast Eur. Design Autom., Comput. Eng., Comput. Netw. Soc. Media Conf.*, 2016, pp. 72–79.
- [21] I. Traore, *Continuous Authentication Using Biometrics: Data, Models, and Metrics: Data, Models, and Metrics*. Hershey, PA, USA: IGI Global, 2011.
- [22] A. Z. Zaidi, C. Y. Chong, Z. Jin, R. Parthiban, and A. S. Sadiq, "Touch-based continuous mobile device authentication: State-of-the-art, challenges and opportunities," *J. Netw. Comput. Appl.*, vol. 191, Oct. 2021, Art. no. 103162.
- [23] V. Kumar and S. Ray, "Continuous behavioral authentication system for IoT enabled applications," in *Proc. Int. Conf. Netw. Security Blockchain Technol.*, 2021, pp. 51–63.
- [24] S. Eberz, K. B. Rasmussen, V. Lenders, and I. Martinovic, "Evaluating behavioral biometrics for continuous authentication: Challenges and metrics," in *Proc. ACM Asia Conf. Comput. Commun. Security*, 2017, pp. 386–399.
- [25] S. S. Almoahamed, J. A. Clark, and J. Law, "Behaviour-based biometrics for continuous user authentication to industrial collaborative robots," in *Proc. Int. Conf. Inf. Technol. Commun. Security*, 2021, pp. 185–197.

- [26] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *Proc. Int. Conf. Theory Appl. Cryptogr. Tech.*, 2001, pp. 93–118.
- [27] S. A. Kakvi, K. M. Martin, C. Putman, and E. A. Quaglia, "SoK: Anonymous credentials," in *Proc. Int. Conf. Res. Security Stand.*, 2023, pp. 129–151.
- [28] L. Hanzlik and D. Slamanig, "With a little help from my friends: Constructing practical anonymous credentials," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2021, pp. 2004–2023.
- [29] L. Zhang, Y. Ye, and Y. Mu, "Multiauthority access control with anonymous authentication for personal health record," *IEEE Internet Things J.*, vol. 8, no. 1, pp. 156–167, Jan. 2021.
- [30] X. Yao, H. Liu, H. Ning, L. T. Yang, and Y. Xiang, "Anonymous credential-based access control scheme for clouds," *IEEE Cloud Comput.*, vol. 2, no. 4, pp. 34–43, Jul./Aug. 2015.
- [31] T. Li, H. Wang, D. He, and J. Yu, "Designated-verifier aggregate signature scheme with sensitive data privacy protection for permissioned blockchain-assisted IIoT," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 4640–4651, 2023.
- [32] Y. Liu, D. He, M. Luo, H. Wang, and Q. Liu, "ATRC: An anonymous traceable and revocable credential system using blockchain for VANETs," *IEEE Trans. Veh. Technol.*, vol. 73, no. 2, pp. 2482–2494, Feb. 2024.
- [33] D. Rathee, G. V. Policharla, T. Xie, R. Cottone, and D. Song, "ZEBRA: SNARK-based anonymous credentials for practical, private and accountable on-chain access control," *Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2022/1286*, 2022.
- [34] O. Mir, B. Bauer, S. Griffy, A. Lysyanskaya, and D. Slamanig, "Aggregate signatures with versatile randomization and issuer-hiding multi-authority anonymous credentials," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2023, pp. 30–44.
- [35] L. Zhang, D. Mao, Z. Chen, Y. Song, L. Chen, and Z. Qin, "Smart grid data aggregation scheme based on local differential privacy," in *Proc. IEEE Int. Conf. Trust, Security Privacy Comput. Commun. (TrustCom)*, 2022, pp. 1534–1539.
- [36] J. Camenisch and A. Lysyanskaya, "Dynamic accumulators and application to efficient revocation of anonymous credentials," in *Proc. 22nd Annu. Int. Cryptol. Conf.*, 2002, pp. 61–76.
- [37] O. Mir, D. Slamanig, B. Bauer, and R. Mayrhofer, "Practical delegatable anonymous credentials from equivalence class signatures," *Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2022/680*, 2022. [Online]. Available: <https://eprint.iacr.org/2022/680>
- [38] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Security*, 2001, pp. 514–532.
- [39] G. Fuchsbauer, C. Hanser, and D. Slamanig, "Structure-preserving signatures on equivalence classes and constant-size anonymous credentials," *J. Cryptol.*, vol. 32, pp. 498–546, Apr. 2019.
- [40] S. Faust, M. Kohlweiss, G. A. Marson, and D. Venturi, "On the non-malleability of the Fiat-Shamir transform," in *Proc. 13th Int. Conf. Cryptol. India*, 2012, pp. 60–79.
- [41] C. Hanser and D. Slamanig, "Structure-preserving signatures on equivalence classes and their application to anonymous credentials," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Security*, 2014, pp. 491–511.
- [42] X. Wang, C. Zhu, and Z. Liu, "A universally composable linkable ring signature supporting stealth addresses," *Mathematics*, vol. 12, no. 3, p. 491, 2024.
- [43] Z. Bao, D. He, H. Wang, M. Luo, and C. Peng, "A group signature scheme with selective linkability and traceability for blockchain-based data sharing systems in E-health services," *IEEE Internet Things J.*, vol. 10, no. 23, pp. 21115–21128, Dec. 2023.
- [44] O. Mir, D. Slamanig, B. Bauer, and R. Mayrhofer, "Practical delegatable anonymous credentials from equivalence class signatures," in *Proc. Privacy Enhanc. Technol. Symp.*, 2023, pp. 488–513.
- [45] J. Meers and D. Riepel, "CCA secure updatable encryption from non-mappable group actions," *Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2024/499*, 2024. [Online]. Available: <https://eprint.iacr.org/2024/499>
- [46] M. Duparc, T. B. Fouotsa, and S. Vaudenay, "SILBE: An updatable public key encryption scheme from lollipop attacks," *Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2024/400*, 2024. [Online]. Available: <https://eprint.iacr.org/2024/400>
- [47] C. A. Haidar, A. Passelègue, and D. Stehlé, "Efficient updatable public-key encryption from lattices," *Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2023/1400*, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1400>
- [48] Y. Liu, D. He, Q. Feng, M. Luo, and K.-K. R. Choo, "PERCE: A permissioned redactable credentials scheme for a period of membership," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 3132–3142, 2023.
- [49] I. Ali, T. Lawrence, and F. Li, "An efficient identity-based signature scheme without bilinear pairing for vehicle-to-vehicle communication in VANETs," *J. Syst. Archit.*, vol. 103, Feb. 2020, Art. no. 101692.
- [50] S. Setty, "Spartan: Efficient and general-purpose zkSNARKs without trusted setup," in *Proc. Annu. Int. Cryptol. Conf.*, 2020, pp. 704–737.
- [51] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *Proc. IEEE Symp. Security Privacy (SP)*, 2018, pp. 315–334.
- [52] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward, "Aurora: Transparent succinct arguments for R1CS," in *Proc. 38th Annu. Int. Conf. Theory Appl. Cryptogr. Tech.*, 2019, pp. 103–128.
- [53] N. Begum and T. Nakanishi, "An accumulator-based revocation in delegatable anonymous credentials," in *Proc. 8th Int. Symp. Comput. Netw. Workshops (CANDARW)*, 2020, pp. 314–320.
- [54] J. Liang, Z. Qin, S. Xiao, L. Ou, and X. Lin, "Efficient and secure decision tree classification for cloud-assisted online diagnosis services," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 4, pp. 1632–1644, Jul./Aug. 2021.
- [55] C. Zhang, X. Luo, J. Liang, X. Liu, L. Zhu, and S. Guo, "POTA: Privacy-preserving online multi-task assignment with path planning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 5999–6011, May 2024.
- [56] J. Liang, Z. Qin, J. Ni, X. Lin, and X. Shen, "Practical and secure SVM classification for cloud-based remote clinical decision services," *IEEE Trans. Comput.*, vol. 70, no. 10, pp. 1612–1625, Oct. 2021.
- [57] C. Zhang, M. Zhao, J. Liang, Q. Fan, L. Zhu, and S. Guo, "Nano: Cryptographic enforcement of readability and editability governance in blockchain databases," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3439–3452, Jul./Aug. 2024.
- [58] L. Peng, W. Feng, Z. Yan, Y. Li, X. Zhou, and S. Shimizu, "Privacy preservation in permissionless blockchain: A survey," *Digit. Commun. Netw.*, vol. 7, no. 3, pp. 295–307, 2021.